

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

PENGANTAR MIKROKONTROLER

Lucky Adhi Suardhana - 5024241095

2026

1 Langkah-Langkah Percobaan

Pada percobaan satu, disiapkan satu project PlatformIO, lalu mengonfigurasi file `platformio.ini` dengan target board `blackpill_f401cc`, framework `arduino`, dan upload protocol `stlink`. Setelah menghubungkan pin SWDIO, SWCLK, GND, dan 3.3V dari ST-Link ke board STM32 Blackpill, praktikan menulis program Blink yang memanfaatkan pin `PC13` untuk menyalakan LED secara periodik, kemudian melakukan proses Build dan Upload ke perangkat keras.

Percobaan kedua dimulai dengan mengubah konfigurasi pada `platformio.ini` menggunakan board `esp32dev` dengan framework ESP-IDF. Selanjutnya, file kode sumber utama diganti menjadi `main.c` untuk menulis program berbasis komponen native FreeRTOS yang menggunakan fungsi titik masuk `app_main()`, pengendali level pin `gpio_set_level()`, serta penundaan non-blok `vTaskDelay()`, sebelum akhirnya mengunggah program tersebut langsung ke board ESP32 melalui kabel USB.

Pada percobaan ketiga, praktikan merakit dua pushbutton pada board ESP32 menggunakan resistor eksternal $10\,\mathrm{k}\Omega$ untuk membaca status logika pin melalui fungsi `digitalRead()` pada framework Arduino. Setelah menguji dan mencatat hasilnya di Serial Monitor, praktikan melepas seluruh resistor eksternal tersebut dan mengaktifkan fitur resistor internal melalui parameter `INPUT_PULLUP` serta `INPUT_PULLDOWN` di dalam kode program untuk membandingkan konsistensi pembacaan status tombol.

Percobaan keempat praktikan menghubungkan sebuah pushbutton ke pin GPIO 25 dengan mode `INPUT_PULLUP` pada board ESP32. Di dalam kode program, praktikan membandingkan dua logika penghitung, yaitu `counterNoDebounce` yang langsung mencatat setiap transisi sinyal dan `counterWithDebounce` yang menerapkan fungsi penundaan waktu stabil (*debouncing*), untuk menganalisis perbedaan jumlah hitungan akibat getaran mekanik saat tombol ditekan.

Percobaan kelima dilakukan perakitan sensor LDR bersama sebuah resistor $10\,\mathrm{k}\Omega$ membentuk rangkaian pembagi tegangan, lalu menghubungkan titik tengahnya ke pin input analog GPIO 34 pada board ESP32 yang mendukung ADC (*Analog-to-Digital Converter*). Praktikan kemudian mengunggah program yang membaca nilai mentah 12-bit (0 hingga 4095) menggunakan fungsi `analogRead()` dan mengonversinya menjadi besaran tegangan aktual di Serial Monitor sambil menguji perubahan intensitas cahaya pada sensor.

2 Analisis Hasil Percobaan

Pada percobaan pertama, proses mengirim program Blink ke board STM32 Blackpill berhasil dilakukan menggunakan programmer eksternal ST-Link melalui protokol SWD, yang terbukti karena LED onboard pada pin `PC13` berkedip secara berkala. Keberhasilan ini menunjukkan bahwa PlatformIO dan framework Arduino Core untuk STM32 mampu mengabstraksikan perangkat keras secara baik, sehingga fungsi dasar seperti `pinMode()` dan `digitalWrite()` bisa langsung menjalankan instruksi *active* LOW pada register fisik tanpa perlu konfigurasi manual register GPIO yang rumit.

Percobaan kedua menunjukkan bahwa menggunakan ESP32 dengan framework native ESP-IDF memberikan tingkat pengendalian sistem yang lebih dalam dibandingkan menggunakan framework Arduino. Di sini, titik awal program menggunakan fungsi `app_main()` yang dijalankan di atas sistem operasi *real-time* FreeRTOS. Dengan menggunakan fungsi dasar seperti `gpio_set_level()` dan penundaan non-blok menggunakan `vTaskDelay()`, LED yang terpasang berhasil diatur secara efisien tanpa mengganggu atau menghentikan proses atau tugas lain yang berjalan di latar belakang mikro-

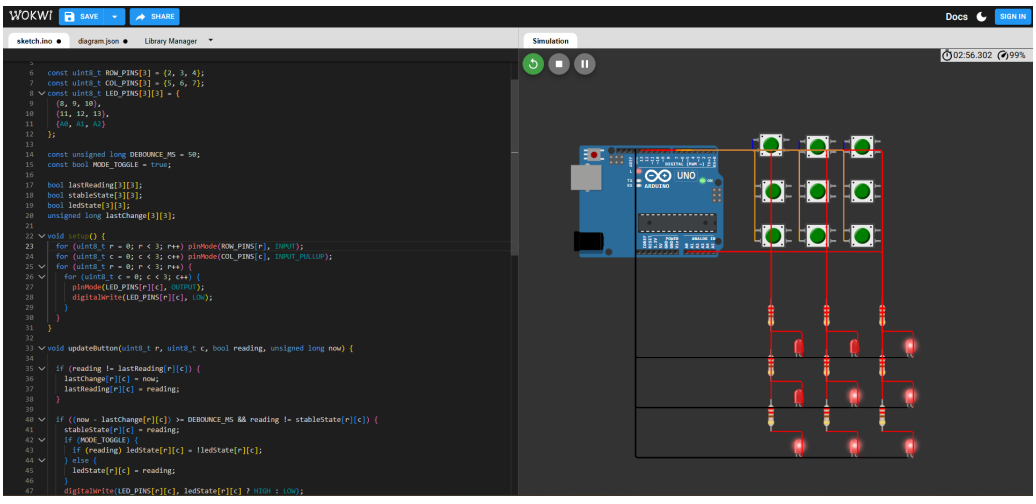
kontroler.

Hasil pengujian pada percobaan ketiga menunjukkan bahwa penggunaan resistor *pull-up* dan *pull-down*, baik yang dipasang secara eksternal maupun diaktifkan secara internal menggunakan fungsi `INPUT_PULLUP` dan `INPUT_PULLDOWN`, berhasil menjaga pin GPIO agar tidak berada dalam kondisi *floating* saat tombol tidak ditekan. Melalui pemantauan Serial Monitor, terbukti bahwa konfigurasi *pull-up* menghasilkan logika HIGH saat tombol diam dan LOW saat ditekan, sedangkan konfigurasi *pull-down* memberikan logika sebaliknya (LOW saat diam dan HIGH saat ditekan), sehingga status tombol dapat terbaca secara konsisten dan akurat.

Percobaan keempat membuktikan bahwa kontak mekanik pada pushbutton memunculkan fenomena *bouncing* (transisi sinyal HIGH-LOW berulang akibat getaran fisik sesaat), yang menyebabkan penghitung tanpa penyaring (`counterNoDebounce`) bertambah secara keliru hanya dari satu kali penekanan. Dengan menerapkan algoritma penundaan waktu stabil (*debouncing*) sebesar $50\,\mathrm{ms}$, sistem berhasil memvalidasi jeda sinyal sehingga penghitung dengan penyaring (`counterWithDebounce`) hanya bertambah tepat satu kali per aksi tekan, menegaskan pentingnya *debouncing* dalam menjaga keandalan pembacaan logika input digital.

Dan pada percobaan kelima, percobaan menunjukkan fungsi ADC (*Analog-to-Digital Converter*) sebagai mode akses GPIO ketiga yang mampu menerjemahkan perubahan besaran fisik kontinu, berupa variasi resistansi LDR akibat intensitas cahaya, menjadi rentang nilai digital diskret 12-bit (0 hingga 4095). Melalui rangkaian pembagi tegangan, nilai mentah tersebut dapat dikonversi secara akurat menjadi parameter tegangan aktual menggunakan rumus matematis, yang membuktikan keunggulan ADC dalam merepresentasikan spektrum kondisi yang jauh lebih luas dibandingkan sekadar status biner HIGH atau LOW pada input digital.

3 Hasil Tugas Modul



Gambar 1: Wiring Simulasi Pushbutton 3x3

1. Bouncing terjadi pada tiap saklar secara individual, bukan karena banyaknya tombol yang ditekan. Kontak logam di dalam tombol bersifat elastis. Saat tombol ditekan, kedua kontak bertumbukan, memantul, dan bertumbukan lagi beberapa kali sebelum diam. Proses ini berlangsung sekitar 1–$20\,\mathrm{ms}$ dan terjadi lagi saat tombol dilepas. Secara listrik, satu tekanan fisik muncul di pin kolom sebagai rangkaian transisi HIGH–LOW–HIGH–LOW yang sangat cepat.

Mikrokontroler membaca jauh lebih cepat daripada durasi bouncing tersebut. Satu siklus scan matrix hanya membutuhkan puluhan mikrodetik hingga sekitar $1\,\mathrm{ms}$, sehingga setiap pantulan terbaca sebagai peristiwa tekan dan lepas yang terpisah. Akibatnya, satu tekanan fisik terbaca sebagai banyak tekanan.

2. Rancangan ini menggunakan konfigurasi **pull-up**. Pin kolom dikonfigurasi `INPUT_PULLUP`, yaitu resistor internal sekitar 20–$50\,\mathrm{k\Omega}$ yang menarik pin ke V_{CC}. Tombol menghubungkan kolom ke baris, dan baris yang sedang di-scan diberi tegangan LOW.

Kondisi	**Pin kolom**
Tombol tidak ditekan	HIGH (ditarik ke V_{CC} oleh pull-up)
Tombol ditekan dan barisnya sedang di-scan (LOW)	LOW (tersambung ke baris $0\,\mathrm{V}$)
Tombol ditekan tetapi barisnya tidak di-scan (INPUT, high-Z)	HIGH (tidak ada jalur ke GND)

Tabel 1: Kondisi pin kolom pada konfigurasi pull-up.

Logika pembacaannya bersifat *active-low*: `digitalRead() == LOW` berarti tombol ditekan. Saat tombol menutup, jalur ke GND melalui pin baris (output LOW) jauh lebih kuat daripada pull-up, sehingga tegangan kolom turun mendekati $0\,\mathrm{V}$. Arus yang mengalir kecil, sekitar $0{,}1$–$0{,}25\,\mathrm{mA}$. Input CMOS memiliki impedansi yang sangat tinggi. Tanpa pull-up, tegangan pin tidak ditentukan oleh rangkaian, melainkan oleh derau yang masuk, misalnya kopling kapasitif dari kabel di dekatnya, medan listrik jala-jala $50\,\mathrm{Hz}$, atau sentuhan tangan. Akibatnya:

- **Pembacaan acak:** tegangan melayang di sekitar ambang logika (pada ATmega328P dengan $V_{CC} = 5\,\mathrm{V}$, di bawah sekitar $1{,}5\,\mathrm{V}$ terbaca LOW dan di atas sekitar $3\,\mathrm{V}$ terbaca HIGH). Hasil `digitalRead` dapat berubah sendiri dan menghasilkan "tekanan hantu" tanpa ada tombol yang ditekan.
- **Arus berlebih:** buffer input yang tegangannya berada di zona tengah dapat menarik arus lebih besar.

Pull-up memberi kondisi awal yang pasti (HIGH), sehingga satu-satunya hal yang mengubahnya adalah tombol yang benar-benar ditekan. Pull-up juga mencegah hubung singkat: menyambung tombol langsung ke V_{CC} tanpa resistor akan menjadi korsleting saat tombol ditekan.

Tabel 2: Perbandingan pembacaan digital dan analog.

	Digital input (tombol)	**Analog input (ADC)**
Hasil	1 bit: HIGH atau LOW	Bilangan bulat n-bit: 0 sampai $2^n - 1$
Cara kerja	Buffer input membandingkan tegangan dengan ambang tetap	Tegangan di-sample, lalu dibandingkan bertahap dengan tegangan referensi (successive approximation)
Jumlah kondisi	2	Arduino Uno 10-bit: $2^{10} = 1024$ level (resolusi $\approx 4{,}9\,\mathrm{mV}$ pada $5\,\mathrm{V}$); ESP32 12-bit: 4096 level
Tegangan di antara ambang	Tidak terdefinisi	Tetap terbaca sebagai angka

3. Tegangan di pin pada dasarnya kontinu. Pin digital membuang hampir seluruh informasinya dan hanya menyisakan jawaban apakah tegangan berada di atas atau di bawah ambang. ADC mengukur tegangan tersebut terhadap referensi (AREF atau V_{CC}), membaginya menjadi 2^n interval, dan melaporkan interval yang sesuai. Itulah sebabnya ADC dapat merepresentasikan lebih dari dua kondisi. Untuk ADC 10-bit dengan referensi V_{ref}, nilai yang terbaca adalah

$$\text{ADC} = \left\lfloor \frac{V_{in}}{V_{ref}} \cdot 1024 \right\rfloor. \tag{1}$$

Besaran fisik LDR adalah resistansi, bukan tegangan. Resistansinya berubah kontinu terhadap cahaya, kira-kira berorde $\mathrm{M}\Omega$ saat gelap dan beberapa $\mathrm{k}\Omega$ saat terang. Agar dapat dibaca ADC, LDR dipasang bersama resistor tetap R sebagai pembagi tegangan (LDR ke V_{CC}, R ke GND):

$$V_{out} = V_{CC}\,\frac{R}{R + R_{LDR}}. \tag{2}$$

Semakin terang, R_{LDR} semakin kecil dan V_{out} semakin besar. Dengan pin digital, LDR hanya berfungsi sebagai detektor terang/gelap dengan satu ambang. Dengan ADC, program dapat membaca tingkat cahaya, menentukan ambang sendiri, atau melakukan kalibrasi.

(a) **LDR tidak cocok ditempatkan di dalam matrix.** Tombol hanya memiliki dua keadaan, terbuka atau tertutup. LDR selalu menghubungkan baris dan kolom dengan resistansi tertentu, sehingga kolom tidak pernah benar-benar HIGH atau LOW dan terus mengganggu pembacaan tombol lain pada kolom yang sama. LDR sebaiknya dipasang di luar matrix, pada pin analog tersendiri dengan pembagi tegangan.

(b) **Pin kolom dan baris pada rancangan ini bukan pin ADC.** Pada Arduino Uno hanya A0–A5 yang memiliki ADC. Pada rancangan ini A0–A2 sudah dipakai LED, sehingga A3–A5 masih tersedia untuk LDR.

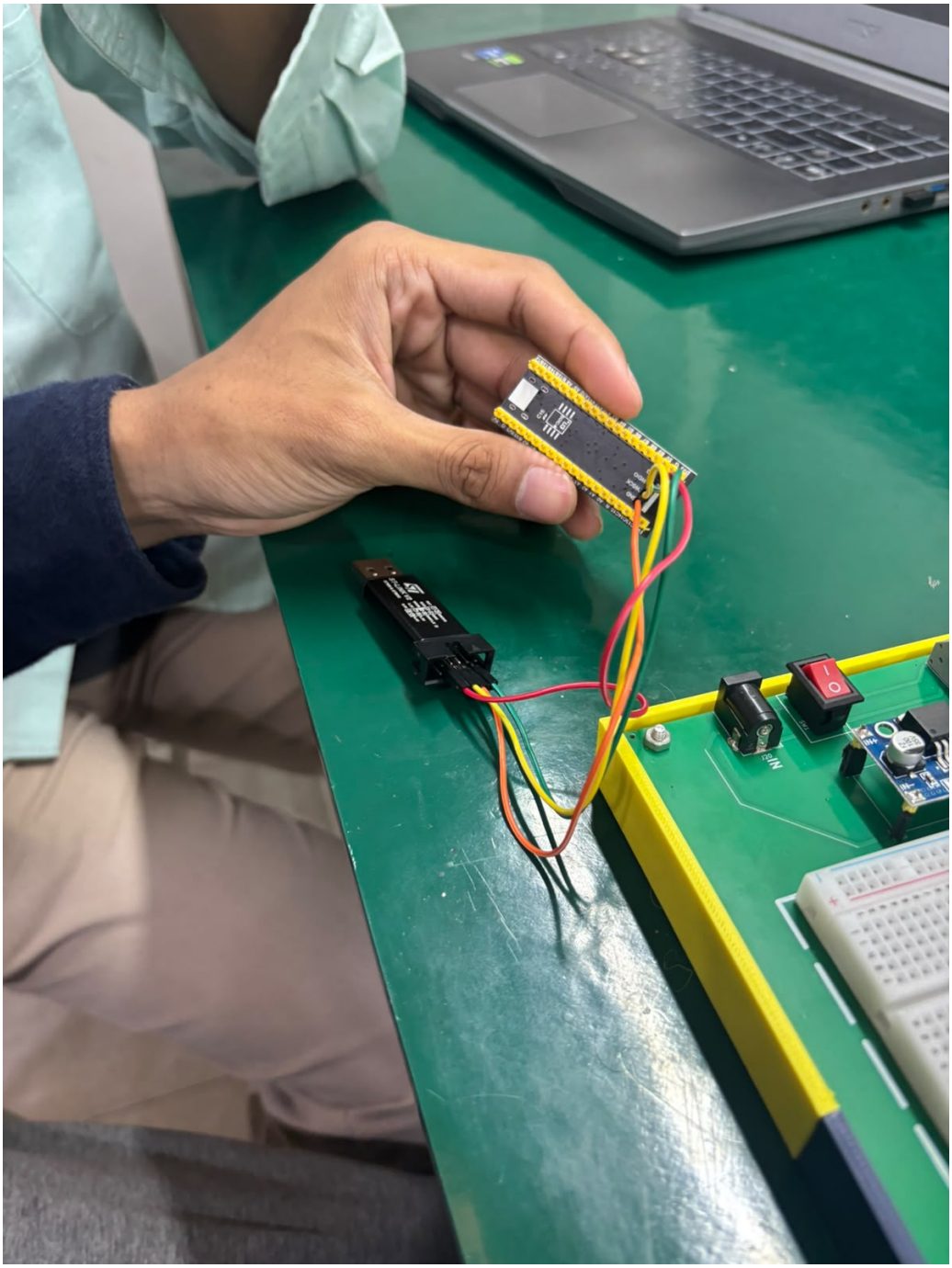
Pembacaan ADC tidak memerlukan debouncing seperti tombol, tetapi memerlukan penanganan lain: derau pada nilai ADC biasanya diatasi dengan merata-ratakan beberapa sampel atau filter sederhana. Satu konversi pada Uno juga membutuhkan sekitar $100\,\mu\mathrm{s}$, jauh lebih lama daripada `digitalRead`.

4 Kesimpulan

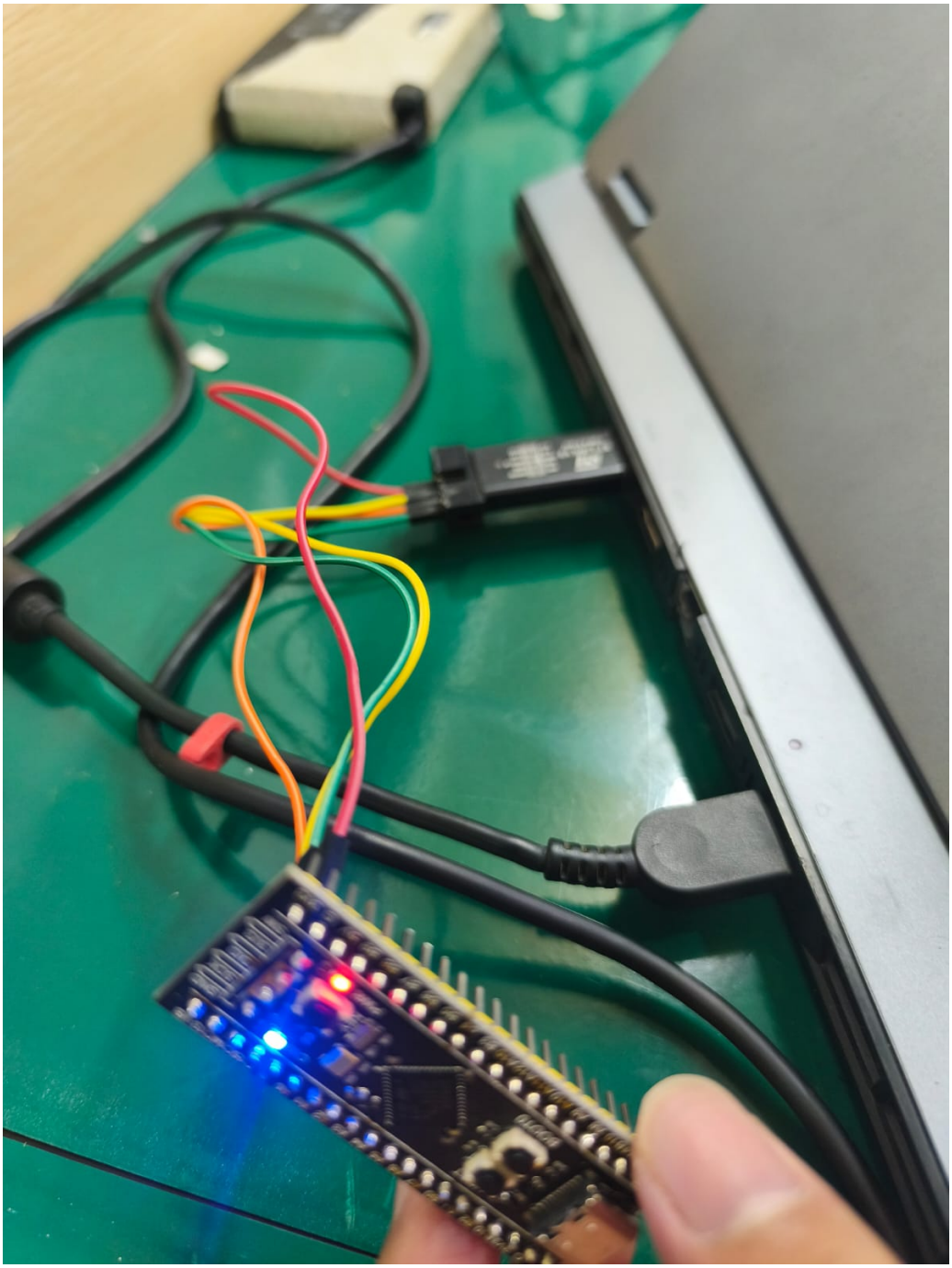
Dengan dilakukannya praktikum ini, praktikan dapat mengenali cara kerja mikrokontroler mulai dari lingkungan pengembangan (PlatformIO) dengan beberapa framework (Arduino dan ESP-IDF), konfigurasi pin, manajemen input digital dan resistor pull, penggunaan algoritma debouncing, dan Akses GPIO Analog melalui ADC.

5 Lampiran

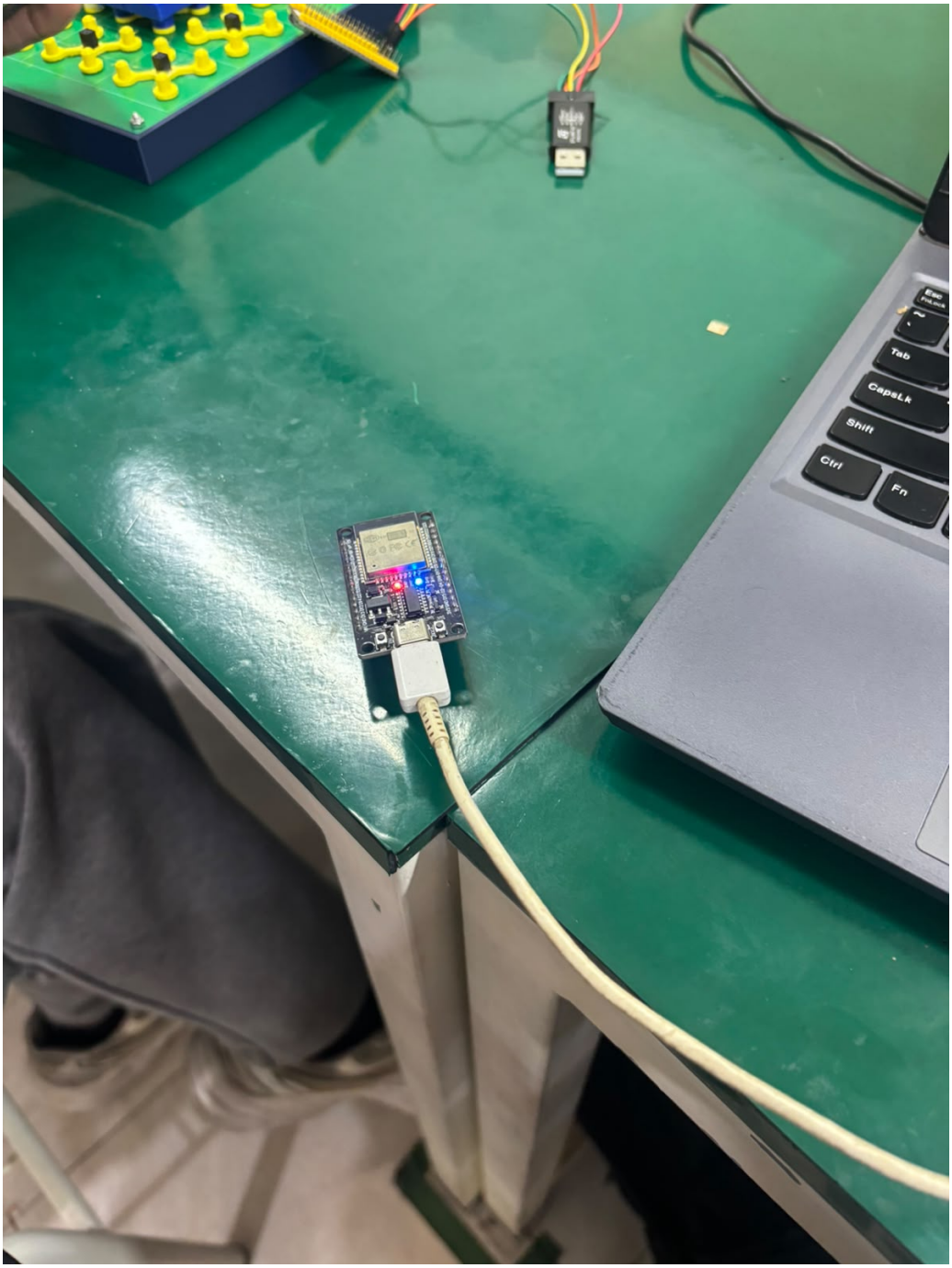
5.1 Dokumentasi saat praktikum



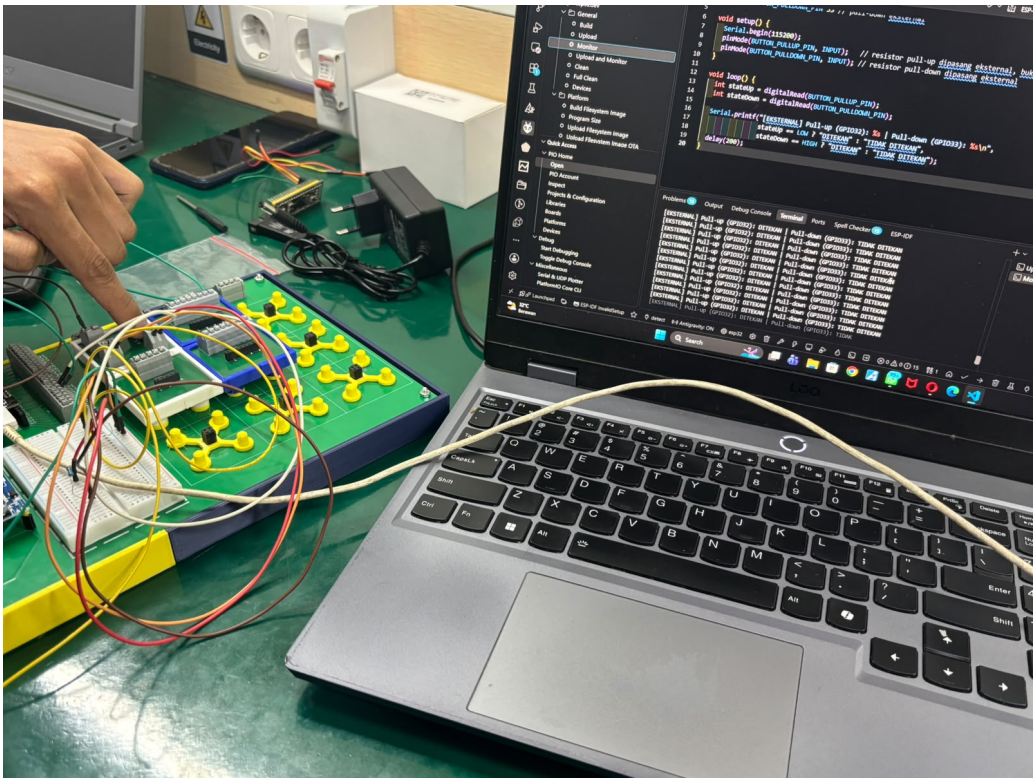
Gambar 2: Wiring Percobaan 1



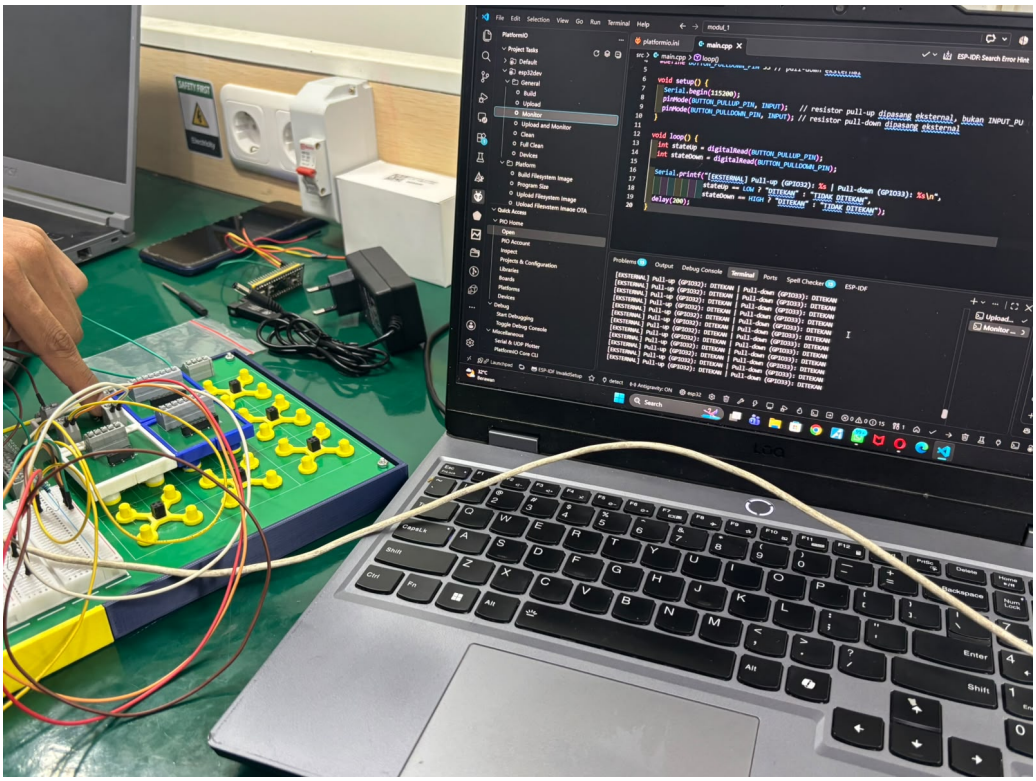
Gambar 3: Hasil Percobaan 1



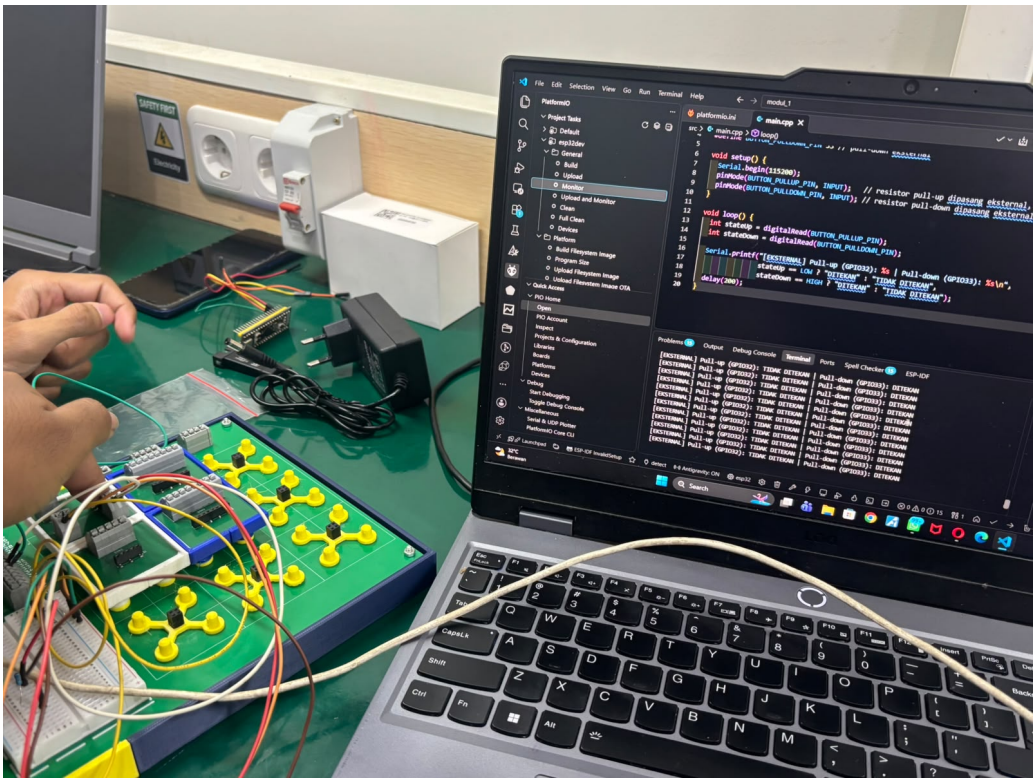
Gambar 4: Hasil Percobaan 2



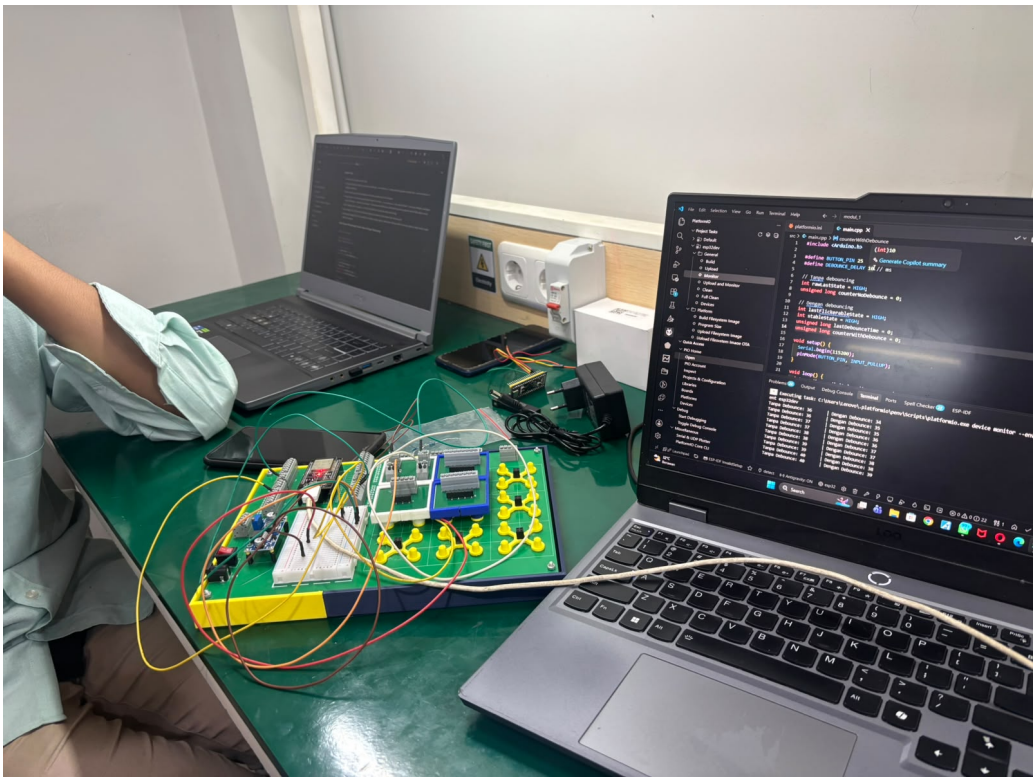
Gambar 5: Hasil percobaan 3



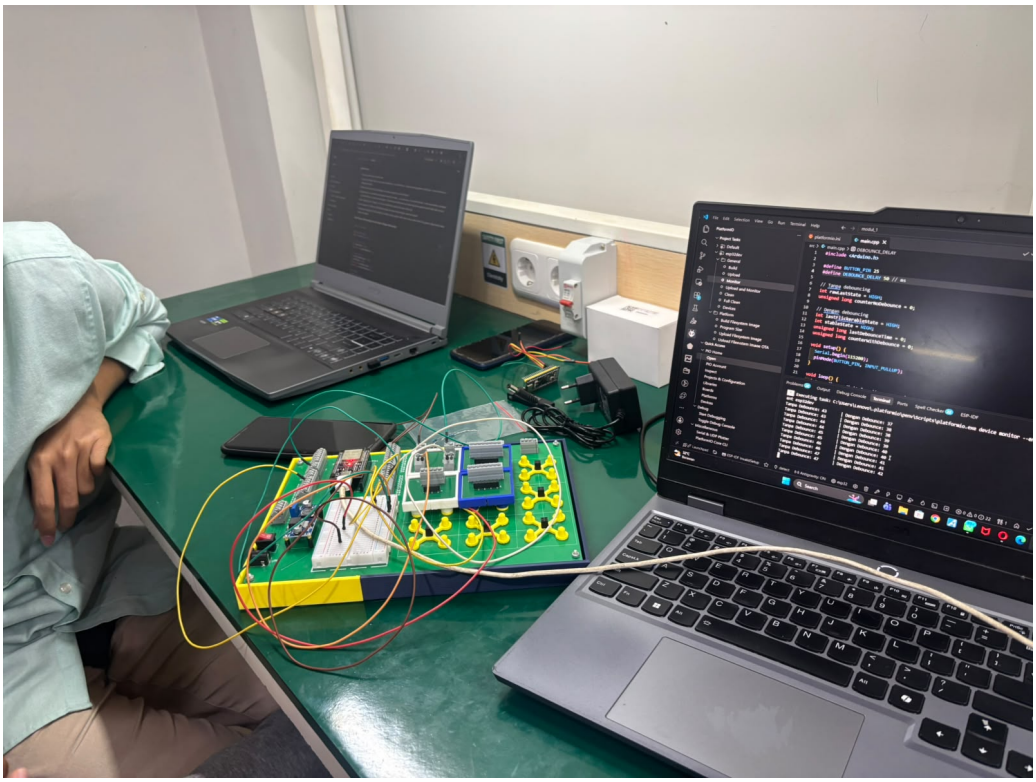
Gambar 6: Hasil percobaan 3 (Pull-down di tekan)



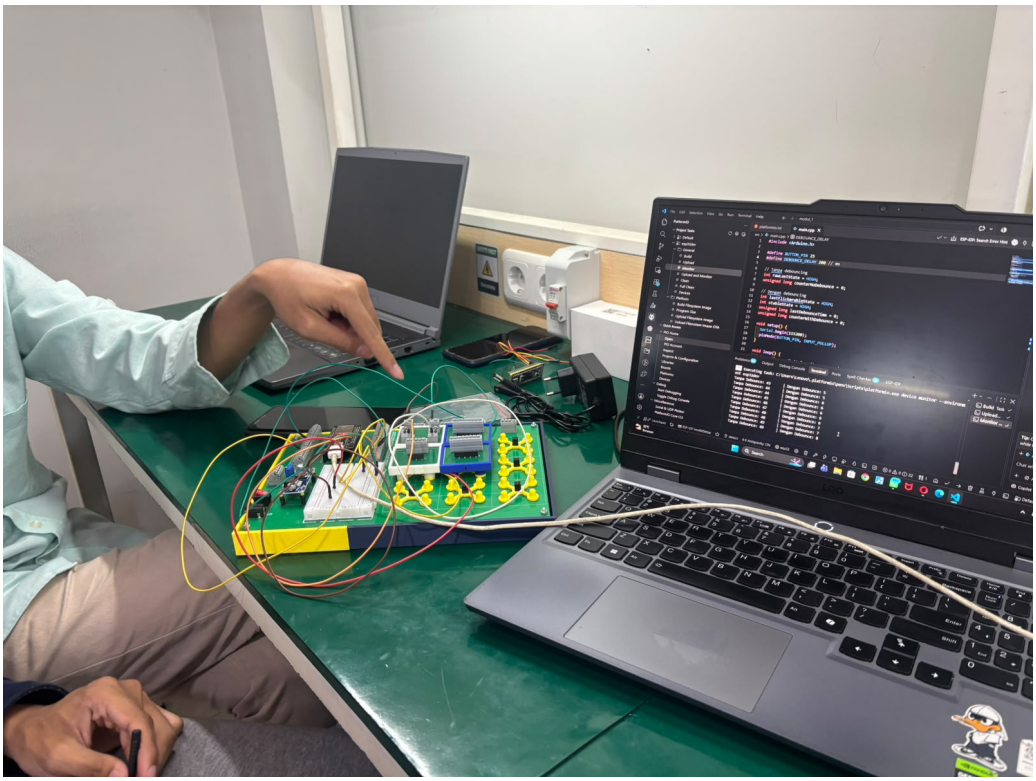
Gambar 7: Hasil percobaan 3 (pull-up di tekan)



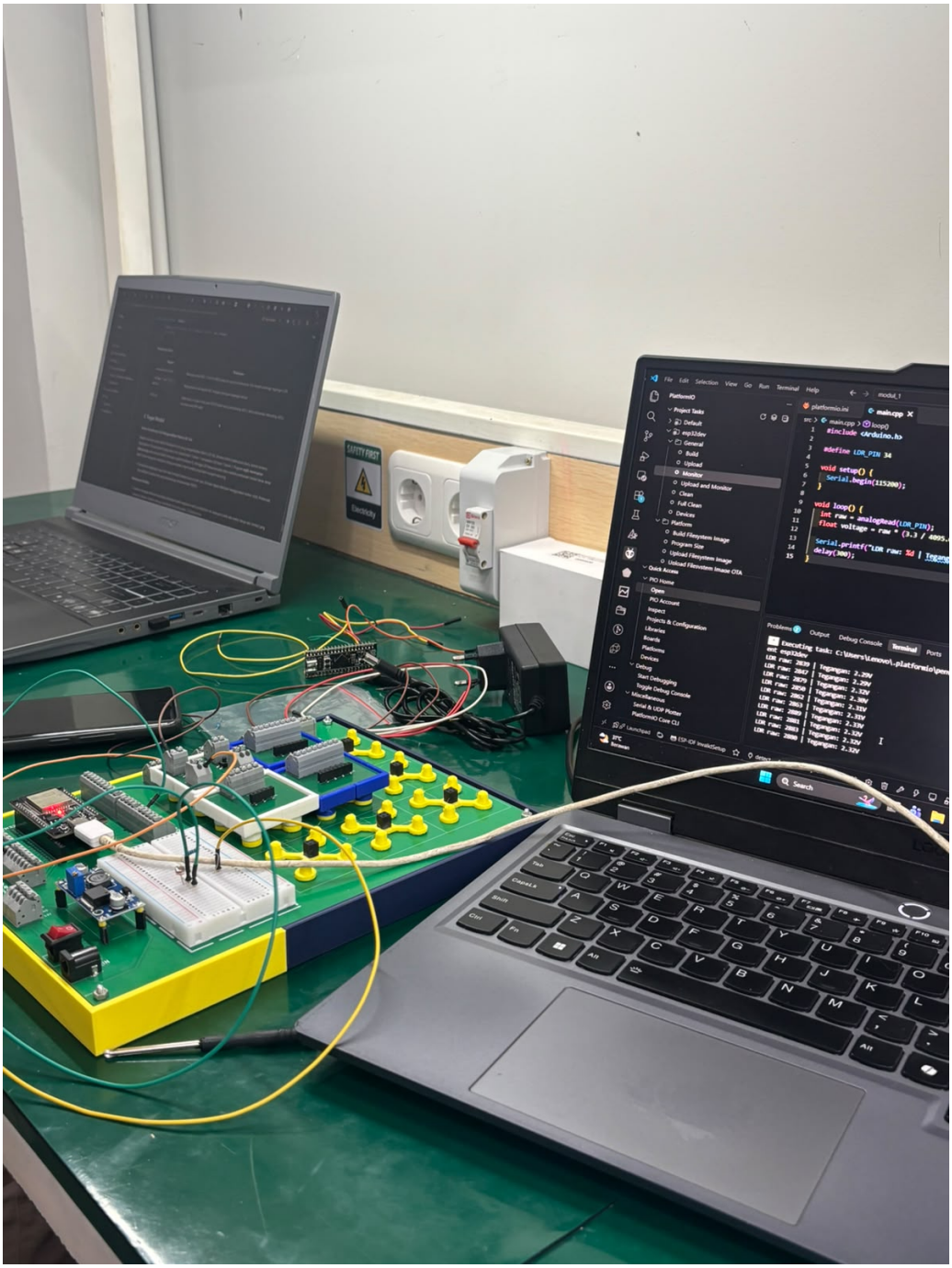
Gambar 8: Hasil percobaan 4 (10ms)



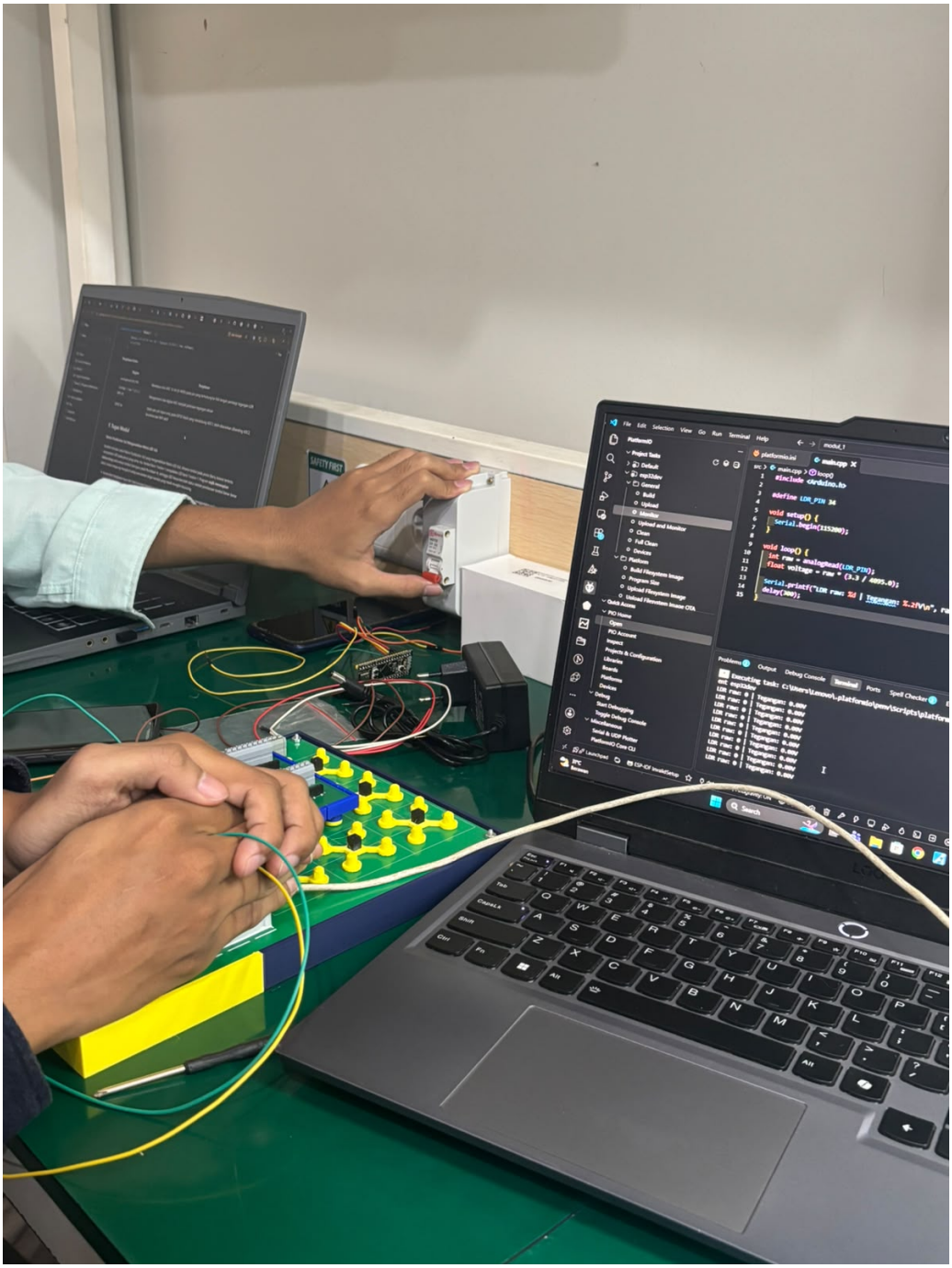
Gambar 9: Hasil percobaan 4 (50ms)



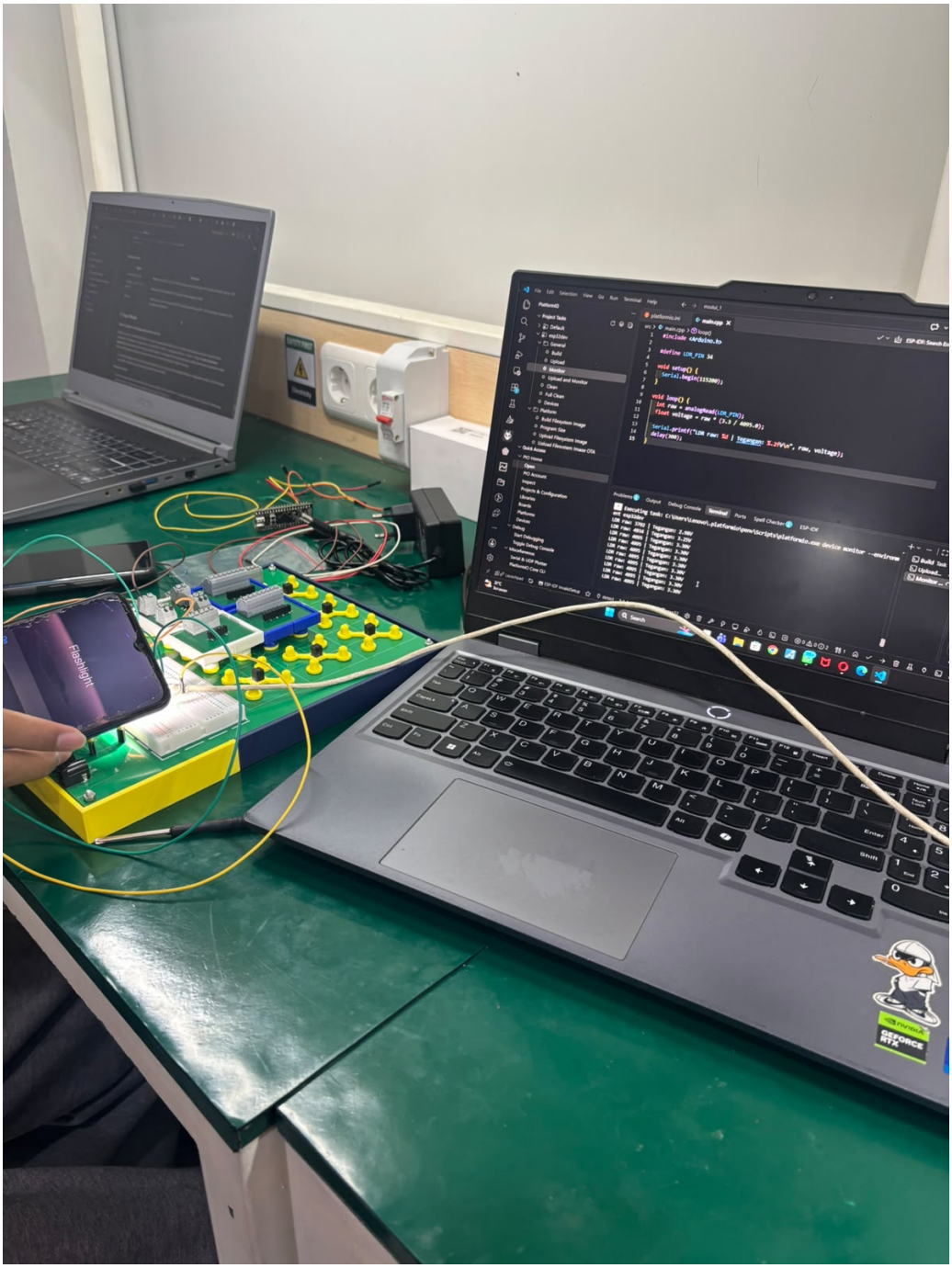
Gambar 10: Hasil percobaan 4 (200ms)



Gambar 11: Hasil percobaan 5



Gambar 12: Hasil percobaan 5 (LDR ditutup)



Gambar 13: Hasil percobaan 1 (LDR disinari)